EduPlay Workflow and Procedure Guide

Step-by-step setup, demonstration, and submission instructions

WORKFLOW AND SUBMISSION GUIDE

Project	EduPlay Smart Educational Media Player
Release baseline	0.1.15 | Source commit add199b
Team	[INSERT TEAM NAME]
Institution and course	[INSERT INSTITUTION AND COURSE]
Supervisor	[INSERT NAME, IF REQUIRED]
Prepared	31 October 2026

Replace all bracketed fields before submission.

How to Use This Guide

Use this guide to prepare the application, run a repeatable demonstration, organize project materials, and submit links and supporting documents. Replace every bracketed field. The repository contains no dataset, research paper, or demo video, so supply those items before final submission.

How the Project Came to Be

The team began with a learning problem: reviewing long lecture videos to find a specific explanation is slow. The development followed an incremental SDLC path, with each stage adding a capability to the desktop player:

1. Identify the problem. Learners needed a practical way to play lecture media, locate spoken topics, and review key ideas without repeatedly scrubbing through an entire recording.

2. Choose the desktop stack. A desktop application was selected for direct media-file use. Electron was chosen because the team already knew HTML, CSS, and JavaScript; that familiarity reduced setup time and supported a cross-platform desktop UI.

3. Build the media player. Implement the Electron playback window and core controls first, creating the base app that the learning features could extend.

4. Add transcription. Integrate Python Whisper to convert speech into an SRT transcript with timestamps. Parse the cues into timed text segments so the application can search them and jump back to the matching video time.

5. Connect Ask AI to the transcript. Run a local Ollama text model and provide it with question-relevant transcript excerpts. The model answers from the lecture context, rather than treating the video question as a generic prompt.

6. Prepare local models and learning context. Configure a text model for language tasks and a vision model for images. Keep transcript cues structured and timestamped; sample long transcripts for text generation and pair selected frames with nearby transcript context for storyboards.

7. Implement and refine study tools. Generate flashcards from transcript segments with source cues, then generate storyboards from selected frames plus transcript context. Validate the returned structure, preserve timestamps, cache results, and test the feature flows.

This sequence is a project-development narrative, not a claim that every feature was completed in a single uninterrupted phase. The current source shows the resulting architecture and behaviors; the team should add dated milestones if the submission requires a formal development timeline.

Submission Links and Project Details

Project source repository: **https://github.com/Drona-Srivastava/EduPlay**

Shared project archive: **[PASTE DRIVE VIEW LINK AFTER UPLOAD]**

Dataset or approved lecture collection: **[PASTE DATASET DRIVE LINK]**

Demo video with all team members: **[PASTE DEMO VIDEO DRIVE LINK]**

Research paper: **[PASTE PAPER DOI OR STABLE LINK]**

Team, institution, and course: **[INSERT VERIFIED DETAILS]**

No dataset is included in the repository. Identify the dataset source, owner, license or permission, format, and approximate size. Share only materials the team is allowed to distribute.

Roles and Responsibilities

Member	Preparation	Demo responsibility
[Member 1]	Confirm objective and speaking order.	Introduce the team, problem, and project goal.
[Member 2]	Verify the cited paper and prepare its summary.	Explain the paper's problem, method, and findings.
[Member 3]	Prepare architecture and source explanation.	Explain the system workflow and implementation.
[Member 4]	Prepare sample video and caption workflow.	Show captions, transcript search, and Ask AI.
[Member 5]	Prepare learning tools and test results.	Show flashcards, storyboard, evidence, and limits.

Combine roles if the team is smaller than five people. Add rows for a larger team. Every member must appear and speak in the final recording.

Prerequisites

- A Windows, macOS, or Linux computer; commands below use Windows PowerShell.
- Node.js and npm; project CI currently targets Node 24.
- Python 3.11 and pip for source development, plus requirements.txt packages.
- Ollama with phi3 and qwen2.5vl:3b installed; model files require several gigabytes of storage and suitable memory.
- An approved lecture file and an existing or generated transcript.
- A microphone, screen recorder, and folder for project deliverables.

Install and Launch on Windows

1. Get the source. Clone the repo or extract the submission ZIP, then open PowerShell in the project directory.

Source commands

```
git clone https://github.com/Drona-Srivastava/EduPlay.git
cd EduPlay
npm ci
```

Source: Project README and package-lock.json

2. Prepare Python. Activate the environment in the same shell used to launch Electron so the caption process can import Whisper and PyTorch.

Python 3.11 commands

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Source: requirements.txt

3. Install local models. Install Ollama for your operating system, pull the text and vision models, and verify them with ollama list. If Ollama is already active, do not start another server.

Ollama commands

```
ollama pull phi3
ollama pull qwen2.5vl:3b
ollama list
```

Source: README.md and server.js defaults

4. Launch EduPlay. Keep the Python environment active and start the desktop app. It starts the local Express backend automatically.

Start the desktop app

```
npm start
```

Source: package.json start script

5. Confirm setup. Verify both models appear in ollama list. If the AI backend is unavailable, check the Ollama service and local status before recording.

Run the Main Workflows

Generate or load captions

1. Open an approved media file in EduPlay.

2. Start caption generation and wait for completion. Whisper writes an SRT file beside the media with suffix _AI_captions_en.srt.

3. Check that the SRT contains numbered cues and start/end timestamps. The current script auto-detects spoken language; it does not force English translation.

4. Alternatively, load an allowed subtitle file and confirm cues display during playback.

Evidence A. Captioned playback

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Insert a screenshot of the actual video and synchronized subtitles.

Search the transcript

1. Search for a word or phrase present in the captions.

2. Select a result and confirm playback seeks to the matching moment.

3. Record the query and timestamp for the demo notes.

Ask AI about the video

1. Wait until the transcript is loaded and ready.

2. Open Ask AI and ask a question answered in the transcript.

3. Show the response and explain its source. Record any error honestly rather than presenting it as a success.

Evidence B. Ask AI

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Insert an actual app screenshot with the question and response.

Generate flashcards

1. Open Learning tools and choose Flashcards.
2. Confirm the Text model is an installed model, normally phi3.
3. Generate cards, reveal an answer, inspect its transcript excerpt, and jump to the source timestamp.
4. Record the actual number of cards and any errors.

Generate a storyboard

1. Open a video and wait for its duration and dimensions to load.
2. Choose Storyboard and confirm the Vision model is installed, normally qwen2.5vl:3b.
3. Generate the storyboard and wait for scene detection and frame analysis.
4. Open a frame, explain the image and transcript context, and jump to its timestamp.

Evidence C. Storyboard

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Insert an actual frame, explanation, transcript context, and timestamp.

Testing and Results Record

Run unit checks

```
npm test
```

Source: package.json test script

Check	Expected evidence	Team result
Unit checks	Pass/fail summary	[INSERT ACTUAL OUTPUT]
Captions	SRT created and subtitles visible	[PASS / FAIL / NOTES]
Search	Match seeks to correct time	[PASS / FAIL / NOTES]
Ask AI	Transcript-based answer displayed	[PASS / FAIL / NOTES]
Flashcards	Cards cite transcript cues	[PASS / FAIL / NOTES]
Storyboard	Frames show context and seek links	[PASS / FAIL / NOTES]

Test environment: [OS, CPU, GPU, RAM, Node, Python, Ollama]. Test media: [file, duration, source permission]. Do not claim accuracy or performance results without explaining how they were measured.

Demo Video Plan

Record a 7–9 minute walkthrough. Every member must appear on camera or in an identified speaking segment and explain an assigned project or paper section.

Time	Speaker	What to show and explain
0:00–0:45	[Member 1]	Introduce the team, learning problem, and project objective.
0:45–2:00	[Member 2]	Explain the paper's motivation, question, method, findings, and citation.
2:00–3:00	[Member 3]	Explain Electron, Whisper/FFmpeg, the local API, and Ollama models.
3:00–5:00	[Member 4]	Open a lecture, generate/load captions, search, and ask a grounded question.
5:00–6:30	[Member 5]	Generate flashcards/storyboard and show a source or timestamp jump.
6:30–8:00	All members	Present observed tests, limitations, conclusion, and next step. Ensure each person speaks.

Paper explanation prompts

- What problem does the paper study, and why does it matter?
- What method or experiment did the authors use?
- What evidence and results did the paper report?
- Which part informed EduPlay, and what did the team implement?
- What limitations should viewers keep in mind?

Approved paper citation: [AUTHORS, TITLE, VENUE, YEAR, DOI OR URL]

Recording checklist

- Every member is introduced and speaks.
- The paper citation and explanation match the source.
- Use media the team may share and hide personal information.
- Keep voice, app text, and cursor readable; do not conceal error states.
- Show real outputs and do not present placeholders as results.
- Export MP4 and play the exported copy before uploading.

Demo video link after upload: [PASTE SHARED DRIVE VIEW LINK]

Organize and Upload the Submission

The project ZIP includes tracked source, tests, documentation, package metadata, and design assets. It excludes node_modules, build outputs, downloaded models, local Python environments, user videos, generated subtitles, and the dataset because these are machine-specific, regenerable, or absent from the repository.

1. Create a Drive folder called EduPlay Submission with Project Archive, Documents, Demo Video, Dataset, and Research Paper subfolders.

2. Upload EduPlay_Submission_Package.zip to Project Archive.

3. Upload the project report and this procedure guide to Documents. Include editable DOCX and PDF copies if accepted by your course.

4. Upload approved data or authorized access instructions to Dataset. Include source, license, format, and permission details.

5. Upload the final MP4 with all team members to Demo Video; play the uploaded copy to verify it.

6. Upload or link the cited paper subject to copyright and course rules.

7. Set permissions as required. Use institution-only sharing where required; use anyone-with-link only when allowed.

8. Copy viewer links into this guide and any required report fields.

9. Open the links as a reviewer or in a private browser window and verify access.

Drive item	Link to paste into this guide
Project archive	[PASTE DRIVE VIEW LINK]
Dataset	[PASTE DRIVE VIEW LINK]
Demo video	[PASTE DRIVE VIEW LINK]
Research paper	[PASTE DRIVE VIEW LINK OR DOI]

Final Submission Checklist

- The ZIP opens and contains source code, design assets, tests, README, and requested documents.
- The report contains actual code excerpts and screenshots from the working app.
- This guide has working archive, dataset, demo, and paper links or approved access instructions.
- The video includes all team members and a correct explanation of the paper.
- Dataset provenance, license, format, and permission are stated.
- The results table reflects the submitted build and observed tests.
- All bracketed fields are replaced, or marked not applicable with a reason.